Blu-Train

someone
Research Department
BluBridge Technologies
Chennai, Tamilnadu
https://blubridge.com/

Abstract

Training attention-based models efficiently on consumer and prosumer GPUs remains an underexplored problem, as most kernel optimization work targets datacenter hardware such as the NVIDIA A100 and H100. We present a collection of hand-tuned CUDA kernels covering the full forward and backward training path of an attention-based model, explicitly optimized for the Ada Lovelace (NVIDIA RTX A6000) and Ampere (NVIDIA RTX 3060) architectures through careful tile size and occupancy tuning. Our kernel library covers six critical operations: fused multi-query attention, layer normalization, GELU activation, loss computation, parallel reduction, and the Adam optimizer. Benchmarked against PyTorch baselines, our kernels achieve speedups of up to **Xx** on Ada Lovelace and **Yx** on Ampere across forward and backward passes. We demonstrate that hardware-specific tuning on non-datacenter GPUs yields substantial performance gains that general-purpose frameworks leave on the table.

1 Introduction

Modern Transformer training pipelines rely on a small set of highly repeated GPU kernels whose performance strongly affects end-to-end training time. While large datacenter accelerators are the usual target for kernel optimization, researchers and smaller teams often train and prototype on prosumer hardware. This paper studies that setting by focusing on a GPT-2-style training workload and examining how hand-tuned CUDA kernels can improve the efficiency of the forward and backward pass on RTX-class GPUs.

The current draft uses this section as a placeholder for the final motivation, problem statement, and contribution summary. In the final version, this section will describe the practical bottlenecks in attention-based model training, explain why consumer and prosumer GPUs deserve separate optimization attention, and summarize the measured speedups obtained by the proposed kernels.

2 Background

Transformer language models are composed of repeated blocks containing attention, normalization, activation, loss, and optimizer operations. In a GPT-2-style model, the attention layer maps token embeddings into query, key, and value projections, computes token-to-token interactions over a fixed context length, and combines the resulting values before passing activations to subsequent feed-forward and normalization layers. Because these operations are executed many times during training, even small improvements in memory traffic, parallel reductions, and tensor-core utilization can compound into meaningful training-time gains.

The experiments in this paper use a 124M-parameter GPT-2-style configuration as a representative workload. Figure 1 provides a high-level architecture view, and Table 1 summarizes the hardware

and run configuration used for the kernel benchmarks. The final draft will expand this section with more detailed architectural context and a clearer connection between the model dimensions and the CUDA kernel design choices.

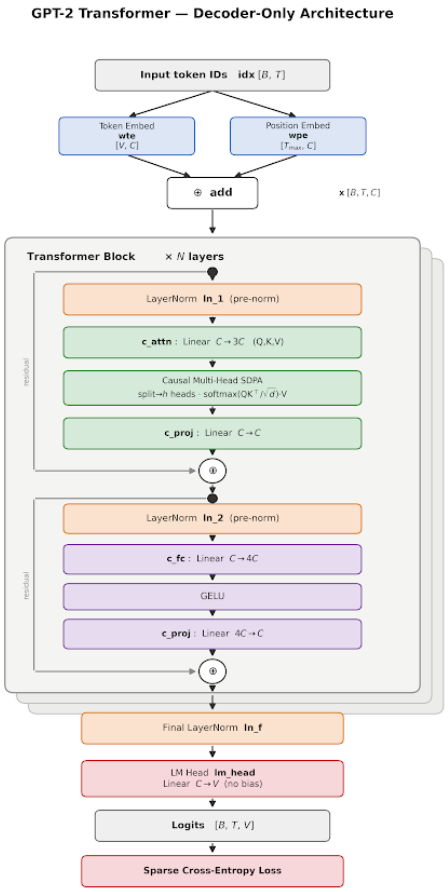

Figure 1: GPT-2 Architecture.

Table 1: Training hardware and hyperparameters for the 124M-parameter GPT-2-style model.

Category	Value
Training hardware	8× RTX 6000 Ada Lovelace GPUs, 48 GB VRAM each
Model size	124M parameters
Mini-batch size (B)	16
Context length (T)	1024
Global batch size	524,288 tokens
Gradient accumulation steps	$524288/(16 \times 1024 \times 8) = 4$
Vocabulary size	50,257 GPT-2 tokens; padded to 50,304
Number of layers (n_{layers})	12
Number of heads (n_{heads})	12
Embedding dimension (n_{embd})	768
Maximum learning rate	6×10^{-4}
Minimum learning rate	$0.1 \times \text{max\_lr} = 6 \times 10^{-5}$
Warmup steps	715
Training dataset	10B-token FineWeb-Edu
Total training steps	19,073
Parallelism strategy	Distributed data parallelism over 8 GPUs

3 Memory Management System

This section is a placeholder for the memory management system used in the Blu-Train kernel stack. The final version will describe how tensors, intermediate activations, scratch buffers, and kernel-specific workspaces are allocated, reused, and synchronized across the training step. It will also explain how memory layout choices interact with the attention, normalization, loss, reduction, and optimizer kernels described later in the paper.

At a high level, the memory management strategy is intended to reduce redundant global-memory traffic, minimize temporary allocations, and keep frequently reused training state close to the GPU execution units whenever possible. The final write-up should discuss buffer lifetimes, alignment requirements, reuse of saved forward-pass statistics during backward computation, and any explicit caching or prefetching mechanisms used to improve throughput on RTX 6000 Ada Lovelace hardware.

This section will also document practical engineering constraints, such as how workspace sizes are chosen for the 124M-parameter GPT-2-style configuration, how the design scales across the eight-GPU distributed data-parallel setup, and which parts of the memory system are specific to the benchmark configuration versus reusable across other model sizes.

4 Kernel Design

4.1 Fused Multi-Head Attention (Forward and Backward)

In the Transformer architecture, the Attention module performs its computation across multiple parallel heads. Each head receives an independent split of the Query, Key, and Value projections, computes its own attention scores, and the results are concatenated to produce the final output — a mechanism known as Multi-Head Attention. Naive implementations of attention kernels are slow primarily due to excessive data movement across the GPU memory hierarchy, heavy inter-warp synchronization, and the absence of operation fusion. Our kernel targets SM89 (Ada Lovelace) and applies optimizations across five categories: arithmetic, memory, algorithm, parallelism, and compiler.

Shared Optimizations (Forward and Backward). On the arithmetic side, we replace `nvcuda::wmma` with explicit PTX `mma.sync.aligned.m16n8k8` instructions, giving direct control over register allocation and instruction scheduling and avoiding unnecessary register spills. All MMA operands use TF32 precision with a `0x1000u` RTNE rounding bias applied to each operand before the MMA call, ensuring correct rounding rather than truncation. On SM89, TF32 tensor cores deliver approximately $8\times$ the throughput of FP32 CUDA cores while retaining full FP32 dynamic range. Fragment loads use `ldmatrix.x4`, a warp-cooperative instruction that loads all four A registers in a

single instruction, avoiding manual register-to-row mapping. Each 16×16 output tile is decomposed into two `m16n8k8` MMA calls, mapping exactly onto the hardware instruction shape and enabling back-to-back scheduling by the compiler. On the memory side, shared memory padding of `HD_PAD = HeadDim + 4` and `BK_PAD = BKN + 4` is applied to all row strides, eliminating bank conflicts that would otherwise serialize warp memory accesses. On the parallelism side, row-wise reductions for softmax use `__shfl_xor_sync` butterfly shuffles across offsets 16, 8, 4, 2, 1 — communicating directly between thread registers with no shared memory traffic and no synchronization barrier. On the compiler side, all kernels are templated on `HeadDim` and tile dimensions, making all tile arithmetic compile-time constants and enabling full loop unrolling. `__launch_bounds__(256, MaxBlocksPerSM)` is specified on both kernels to bound register allocation to the occupancy target. All PTX helper functions carry `__forceinline__` to prevent register allocation breaks across call boundaries. The causal masking flag is a compile-time boolean template parameter, eliminating all causal branches from the non-causal PTX specialization entirely.

Forward Pass. The forward kernel issues all global-to-shared memory transfers via `cp.async.cg.shared.global` — a 16-byte asynchronous DMA instruction that bypasses the register file and copies directly to shared memory without blocking the warp. This enables a double-buffered K prefetch pattern: while the current K tile drives the score GEMM, the next K tile is loaded asynchronously into a second buffer, hiding the $\sim$200–400 cycle global memory latency behind useful computation. Fine-grained pipeline synchronization via `cp.async.commit_group` and `cp.async.wait_group<1>` ensures the minimum necessary synchronization — waiting only for the V load without stalling on the next K prefetch. Online softmax is implemented with running max and running sum statistics maintained across KV tiles, avoiding materializing the full $T \times T$ attention matrix and keeping computation entirely in SRAM. The Ada L2 persistent cache is configured via `cudaAccessPropertyPersisting` to pin the K and V tensors in the 64 MB Ada L2 persisting region — since K and V are read by every Q block but never written, subsequent blocks reuse cached KV tiles without accessing HBM. Dropout is fused into the softmax pass, applying the dropout scale immediately after `expf` with no separate kernel launch or second read of the score matrix. Score GEMM tiles are distributed across warps via round-robin assignment (`tile_idx = warp_id + n * NUM_WARPS`), keeping all warps independent with no inter-warp communication until the final `__syncthreads`.

Backward Pass. The backward kernel uses a KV-tile-centric grid (`dim3(kv_tiles, BH)`), where each block owns one fixed KV tile and iterates over all Q tiles in its inner loop. This design makes dK and dV accumulation entirely atomic-free — no other block touches the same KV rows — at the cost of dQ requiring `atomicAdd`, a deliberate asymmetric tradeoff since dK and dV updates dominate backward pass cost. Register accumulators `dk_acc[8]` and `dv_acc[8]` are held live in registers across the entire Q-tile loop, incurring zero shared memory traffic and zero global atomics for the dominant cost, with the accumulators scattered to global memory exactly once at loop end. A separate precompute kernel evaluates $D_i = \sum_d dO_{i,d} \cdot O_{i,d}$ for all rows before the main kernel launches, reducing redundant D evaluations from $O(T^2)$ to $O(T)$. Softmax probabilities are reconstructed via `exp2f(BWD_LOG2E * (raw_s * scale - L))`, where `BWD_LOG2E` is a compile-time constant — `exp2f` maps to a single `ex2.approx.f32` hardware instruction, eliminating one FP multiply per softmax element versus `expf`. The ds matrix is written simultaneously into two shared memory buffers — row-major for the dQ GEMM and column-major for the dK GEMM — so each subsequent GEMM reads its operand in exactly the layout MMA expects with no transpose pass. Warps are permanently split: warps 0 to `HD_CHUNKS` $-$ 1 handle Phase A GEMMs (QK^T and $dO \cdot V^T$) while warps `HD_CHUNKS` to $2 \cdot$ `HD_CHUNKS` $-$ 1 hold `dk_acc` and `dv_acc` live across the full Q loop, preventing the register pressure doubling that mixed roles would cause. Phase A fuses two independent GEMMs — QK^T and $dO \cdot V^T$ — across four warps simultaneously, halving Phase A wall-clock time. Finally, the staging buffer `tile_st` is reused for both dK and dV final writes via a `__syncthreads` barrier, keeping shared memory footprint at the 43.8 KB target that fits two blocks per SM on Ada Lovelace.

Table 2: Attn FWD (Casual, T = 1024)

Kernel	B	H	D	latency(ms)	HBM GB/s	TFLOPS
BluBridge	16	12	64	0.9344	215.99	27646.24
Pytorch	16	12	64	1.5851	128.67	16469.78

Table 3: Attn BWD (Casual, T = 1024)

Kernel	B	H	D	latency(ms)	HBM GB/s	TFLOPS
BluBridge	16	12	64	2.660352	152.95	24471.70
Pytorch	16	12	64	4.02944	100.00	16001.22

4.2 Layer Normalization (Forward and Backward)

Layer Normalisation is applied at every Transformer block, normalising each token's activation vector to zero mean and unit variance before the affine rescaling by learned parameters γ and β. Naive implementations incur two full passes over the input — one to compute statistics and one to normalise — and suffer from poor occupancy on wide blocks. Our SM89 kernel eliminates the second pass entirely, adapts the block configuration to Ada's register file, and applies targeted vectorisation and reduction optimisations across arithmetic, memory, algorithm, parallelism, and compiler categories.

Shared Optimisations (Forward and Backward). On the parallelism side, both the forward kernel and the backward input-gradient kernel launch **512 threads per block**, up from 256 in the generic SM86 implementation. Ada Lovelace carries a larger per-SM register file than earlier architectures, and the increased thread count fills that resource without spilling, raising occupancy on Ada while leaving the generic path unchanged. All warp-level reductions are implemented with `__shfl_down_sync` butterfly shuffles across offsets 16, 8, 4, 2, 1, communicating partial sums directly between thread registers with no shared memory traffic and no synchronisation barrier at the warp level. On the memory side, all kernel parameters are decorated with `__restrict__`, asserting the absence of pointer aliasing and permitting the compiler to reorder and vectorise loads freely. For `float` inputs, global memory transfers use 128-bit `float4` loads and stores, achieving the maximum memory transaction width and reducing total transaction count by $4\times$ compared to scalar accesses. For `fp16` and `bfloat16` inputs, each `float4` load carries eight packed half-precision values, unpacked in registers via `__half2` and `__nv_bfloat162` reinterpret casts, yielding eight elements per 128-bit transaction. All floating-point accumulation — statistics, softmax denominators, and gradient sums — is performed in `float` regardless of the input dtype, preventing precision loss during reduction. On the compiler side, all kernels carry `__launch_bounds_(512)`, providing the register allocator with a hard upper bound on the thread count and allowing tighter register packing for Ada's occupancy target. The inner loops over input columns carry `#pragma unroll 4`, reducing loop-control overhead by unrolling four iterations per compiler pass. All helper functions (`to_float`, `from_float`, `warpReduceSum`) carry `__forceinline__`, eliminating call frame overhead at every use site. Multi-dtype dispatch is implemented with `if constexpr` blocks, so the `float`, `fp16`, and `bfloat16` code paths are resolved at compile time and generate entirely separate PTX specialisations with no runtime branching.

Forward Pass. The forward kernel implements the **Welford one-pass online algorithm**, computing the row mean μ and variance σ^2 in a single traversal of each row. Each thread accumulates a local count, local sum, and local sum-of-squares as it strides through its assigned columns; the three scalars are then combined using the numerically stable Welford merge formula $\delta = \mu_B - \mu_A,\ \mu' = \mu_A + \delta \cdot n_B/(n_A + n_B)$, ensuring no catastrophic cancellation even for large or ill-conditioned activations. This eliminates the second global memory read of the input that a naive two-pass mean-then-variance implementation would require, halving the input bandwidth cost for the statistics computation. After scalar accumulation, a two-level hierarchical reduction combines partial Welford states into a single block-wide result: warp shuffles reduce each 32-thread warp to a single state, the 16 lane-0 states (512 threads ÷ 32) are written to a 16-entry `s_welford` shared memory buffer, and thread 0 merges all 16 entries sequentially, requiring exactly two `__syncthreads` barriers for the entire statistics phase. The shared memory buffer is sized exactly to the number of warps in the block — 16 entries for 512 threads — with no wasted allocation. The normalisation pass $y = (x - \mu) \cdot r \cdot \gamma + \beta$, where $r = 1/\sqrt{\sigma^2 + \varepsilon}$, is then fused into a single vectorised write pass. For `float` inputs, all four of x, γ, β, and y are accessed as `float4`, computing and storing four output elements per thread per iteration with operands held in registers throughout. The mean and reciprocal standard deviation are saved to global memory once by thread 0 and reused by the backward pass, incurring no recomputation cost.

Backward Pass. The backward computation is split into two structurally distinct sub-kernels, each matched to the access pattern of its output.

The **gamma/beta gradient sub-kernel** (`ln_backward_gamma_beta_sm89_kernel`) performs a column-wise reduction over all rows. Its thread block is a 32×8 2-D tile: 32 threads cover a 32-column slice and 8 threads stride over rows, matching the rectangular access pattern and enabling a natural 8×32 shared memory accumulation tile for `s_dg` and `s_db`. Each thread accumulates its partial $\partial\mathcal{L}/\partial\gamma$ and $\partial\mathcal{L}/\partial\beta$ contributions into registers, writes them to shared memory, and the `ty = 0` row performs a single `atomicAdd` to global memory per column — reducing atomic traffic by $8\times$ compared to per-thread atomics. The grid in the row dimension is sized as $\min(64, \lceil 512/b_x \rceil)$, up from the generic kernel's ceiling of 32, saturating Ada's 142 streaming multiprocessors on wide models. The outer column-block loop carries `#pragma unroll 4` to amortise loop overhead across column tiles.

The **input gradient sub-kernel** (`ln_backward_input_sm89_kernel`) is launched with one block per row and 512 threads, and reads the mean and reciprocal standard deviation saved by the forward pass rather than recomputing them. It makes two vectorised passes over the row: a first pass accumulates the two scalar dot products $s_1 = \sum_d \partial y_d \cdot \gamma_d$ and $s_2 = \sum_d \partial y_d \cdot \gamma_d \cdot \hat{x}_d$ using `float4` loads for `float` inputs; a second pass applies the closed-form gradient $\partial x_d = r\big(\partial y_d \gamma_d - (s_1 + \hat{x}_d \cdot s_2)/N\big)$ and writes ∂x via `float4` stores. Block-level reduction of s_1 and s_2 uses warp shuffles followed by a warp-leaders-only `atomicAdd` pattern: only lane 0 of each warp calls `atomicAdd` into two shared scalars, reducing atomic operations by $32\times$ relative to a per-thread scheme while requiring only one `__syncthreads` between the reduction and the write pass.

Table 4: LayerNorm FWD

Kernel	rows	cols	latency(ms)	HBM GB/s	TFLOPS
BluBridge	16384	768	0.12390	811.66	506.60
Pytorch	16384	768	0.12595	799.67	499.11

Table 5: LayerNorm BWD

Kernel	rows	cols	latency(ms)	HBM GB/s	TFLOPS
BluBridge	16384	768	0.33476	452.01	188.16
Pytorch	16384	768	0.35783	422.36	175.82

4.3 GELU Activation (Forward and Backward)

The GELU activation is applied once per Transformer block, on the $4n_{\text{embd}}$-wide hidden tensor of the feed-forward network - a 16384×3072 activation in the 124M configuration. Like all pointwise activations it has negligible arithmetic intensity: the forward pass reads one element and writes one (8 bytes/element for `float`), and the backward pass reads the upstream gradient and the saved input and writes one gradient (12 bytes/element), against roughly a dozen floating-point operations per element. The operation is therefore entirely *memory-bound*, and the design goal is to reach the DRAM bandwidth ceiling while keeping the transcendental `tanh` evaluation off the critical path. We use the tanh approximation of GELU,

$$\mathrm{GELU}(x) = \tfrac{1}{2}\, x \left(1 + \tanh\left[\sqrt{\tfrac{2}{\pi}}\,(x + 0.044715\, x^3)\right]\right),$$

matching the `approximate='tanh'` mode used by the PyTorch reference model.

Shared Optimisations (Forward and Backward). Both kernels evaluate the hyperbolic tangent with the single hardware instruction `tanh.approx.f32`, emitted via inline PTX (`fast_tanh_sm89`) rather than the multi-instruction `tanhf` library routine, collapsing the dominant arithmetic cost of the activation to one instruction and keeping the FMA pipe clear for the polynomial argument. On the memory side, `float` data is moved in 128-bit `float4` transactions (and `fp16`/`bf16` data in 32-bit `half2`/`__nv_bfloat162` pairs), so each thread loads and stores four (respectively two) elements per

iteration at the maximum transaction width; all pointers are marked `__restrict__` to license this vectorisation. A scalar remainder loop handles the tail when the element count is not a multiple of the vector width. Both forward and backward kernels launch **512 threads per block** (up from 256 in the generic path) to raise occupancy on Ada's larger register file, carry `__launch_bounds__(512)` to bound register allocation to that target, apply `#pragma unroll 8` to the grid-stride loop for deeper instruction-level parallelism, and resolve the `float`/`fp16`/ `bf16` paths at compile time with `if constexpr` so each dtype generates a separate branch-free specialisation. All device helpers (`fast_tanh_sm89`, `gelu_fwd_f32`, `gelu_bwd_f32`, the dtype converters) are `__forceinline__`.

Forward Pass. The forward kernel (`gelu_forward_sm89_kernel`) is a single grid-stride pass: each thread loads a `float4` of inputs, applies `gelu_fwd_f32` to each lane, and writes a `float4` of outputs, with no shared memory and no synchronisation. The grid-stride form lets a launch-bounded grid (capped at 65535 blocks) cover an arbitrarily large activation while each thread retains multiple in-flight `float4` loads under the unrolled loop, saturating the memory pipeline.

Backward Pass. The backward kernel (`gelu_backward_sm89_kernel`) recomputes the GELU derivative from the saved pre-activation input rather than materialising it in the forward pass, trading a few register FMAs - which are free under a memory-bound roofline - for one fewer tensor read. Each thread loads a `float4` of upstream gradients and a `float4` of inputs, computes $\partial x = \partial y \cdot \mathrm{GELU}'(x)$ per lane where GELU' reuses the same single-instruction `tanh.approx.f32` evaluation, and writes a `float4` of input gradients. Unlike the generic backward path, which falls back to scalar accesses, the SM89 backward is fully `float4`-vectorised.

Profiler Evidence. Nsight Compute confirms both kernels are at the memory limit on the RTX 6000 Ada at the 16384×3072 feed-forward shape. The forward kernel sustains **90.27% of DRAM speed-of-light** (822.4 GB/s combined read and write) at $419.1\,\mu$s, and the backward kernel sustains **92.26% of DRAM speed-of-light** (840.6 GB/s) at $676.7\,\mu$s, while compute (SM) throughput stays at 12.75% and 11.04% respectively - the unambiguous signature of a kernel bottlenecked on memory bandwidth rather than the activation arithmetic. The single-shape backward latency measured here ($676.7\,\mu$s) is within 1% of the PyTorch backward at the identical shape ($682.7\,\mu$s).

Table 6: GELU FWD (tanh approximation, 16384×3072 feed-forward activation).

Kernel	rows	cols	latency(ms)	HBM GB/s	DRAM SOL (%)
BluBridge	16384	3072	0.41907	822.41	90.27
Pytorch	16384	3072	–	–	–

Table 7: GELU BWD (tanh approximation, 16384×3072 feed-forward activation).

Kernel	rows	cols	latency(ms)	HBM GB/s	DRAM SOL (%)
BluBridge	16384	3072	0.67674	840.60	92.26
Pytorch	16384	3072	0.68273	833.2	–

4.4 Sparse Cross-Entropy Loss (Forward and Backward)

The cross-entropy loss over a large vocabulary is among the most memory-intensive operations in a language model training step. For a vocabulary of size V and batch size B, a naive implementation reads the full $B \times V$ logit matrix twice - once to find the row maximum and once to compute the exponential sum - and recomputes those statistics a third time during the backward pass. Our implementation eliminates redundant passes through three principal techniques: an online softmax algorithm that fuses the two forward passes into one, asynchronous memory transfers that bypass the register file, and a statistics-reuse protocol that allows the backward pass to skip its reduce kernel entirely.

Shared Optimisations (Forward and Backward). Both kernels issue global-to-shared memory transfers via the PTX instruction `cp.async.cg.shared.global` with a 16-byte transfer width. This instruction bypasses the register file entirely - data travels directly from global memory into shared memory without occupying any registers - and the `.cg` cache qualifier bypasses the L1 data cache,

preventing logit data (which is read sequentially and never reused within a block) from evicting resident activations. Fine-grained pipeline synchronisation is achieved via `cp.async.commit_group` and `cp.async.wait_group 0`, issuing and waiting for exactly one in-flight transfer per iteration with no superfluous stalls. A thread-private staging buffer layout is used in both kernels: each thread t loads four elements into the slots `s_stage`$[4t]$ through `s_stage`$[4t+3]$, a region exclusively owned by that thread. This design deliberately omits `__syncthreads` inside the strided load loop; a barrier inside a variable-trip-count loop would deadlock whenever the vocabulary size is not an exact multiple of $4 \cdot$ `blockDim.x` (e.g. $V = 50{,}304$), because divergent threads would stall at the barrier while others have already exited the loop. Since each thread only ever reads its own four-slot region, coherence is guaranteed without synchronisation. Both kernels implement the **online softmax** algorithm: rather than materialising all V logits and then computing max and sum in separate passes, each thread maintains a running `local_max` and `local_sum` and applies the merge rule

$$\texttt{local\_sum} \leftarrow \texttt{local\_sum} \cdot \exp(\texttt{local\_max} - x) + 1$$

whenever a new maximum x is encountered, rescaling the accumulated partial sum in-place. All arithmetic is performed in `float` regardless of the input storage type, and all kernel pointer parameters carry `__restrict__` to permit the compiler to freely reorder loads. Inner element loops carry `#pragma unroll` to minimise loop-control overhead.

Forward Pass. The forward kernel (`sparse_ce_forward_kernel_vec_save_stats`) launches one block per batch row with 512 threads and a shared memory allocation of $6 \cdot$ `blockDim.x` floats, partitioned into three contiguous regions: a staging buffer of $4 \cdot$ `blockDim.x` floats for in-flight async transfers, followed by `smax` and `ssum` arrays of `blockDim.x` floats each. The three regions are laid out in this order deliberately: placing `smax` and `ssum` *after* the staging buffer ensures the reduction arrays do not alias slots into which `cp.async` DMA operations may still be in flight, preventing a race condition between a thread's reduction write and an in-flight 128-bit transfer targeting the same address. Before entering the vectorised load loop, the kernel performs a 16-byte alignment check on the row pointer; only 16-byte-aligned rows use the asynchronous `float4` path, while unaligned rows fall back to a scalar strided loop. A tail loop handles remainder elements when the vocabulary size is not a multiple of four. After the vectorised pass, thread-local states are written into `smax` and `ssum` and a standard parallel-halving block reduction applies the online softmax merge formula at each step, requiring exactly one `__syncthreads` before the reduction and one per halving stage. Thread 0 then writes the scalar results `saved_max[row]` and `saved_sum[row]` to pre-allocated global scratch tensors held by the autograd node, and computes the per-sample loss as

$$\mathcal{L} = \log(\texttt{final\_sum}) + \texttt{final\_max} - \texttt{logit}[\text{target}],$$

the numerically stable form of cross-entropy that avoids `expf` overflow.

Backward Pass. The backward kernel (`sparseCENormalize_from_stats`) launches one block per batch row with 256 threads and reads `saved_max[row]` and `saved_sum[row]` directly from the forward scratch tensors. This eliminates the `sparseCEReduce_kernel_optimized` stage that the standard two-kernel backward path requires - a full second pass over the $B \times V$ logit matrix to recompute per-row max and sum - saving approximately 3.7 ms per call. The reciprocal `inv_sum` $= 1/$`saved_sum[row]` is precomputed once per block before the load loop, replacing all per-element divisions with a multiply. The kernel then makes a single vectorised pass: `cp.async` transfers four logit values into the staging buffer, and each thread computes

$$\text{prob} = \exp(x - \texttt{final\_max}) \cdot \texttt{inv\_sum}, \qquad \nabla_x = \big(\text{prob} - \mathbf{1}[j = \text{target}]\big) \cdot \text{scale}$$

for each of the four elements. The four gradient values are packed into a register-resident `float4` accumulator and written to global memory as a single 128-bit store, achieving the full memory bus width on the write path. Because all statistics are supplied by the forward pass, no block reduction is performed and no shared memory barriers are required beyond the implicit ordering provided by `cp.async.wait_group 0`.

Table 8: Loss FWD

Kernel	B	vocab	latency(ms)	HBM GB/s	TFLOPS
BluBridge	16384	50304	3.78377	871.17	871.17
Pytorch	16384	50304	8.14745	404.41	404.41

Table 9: Loss BWD

Kernel	B	vocab	latency(ms)	HBM GB/s	TFLOPS
BluBridge	16384	50304	8.22528	800.72	400.36
Pytorch	16384	50304	13.9105	473.99	236.99

4.5 Reduce Kernel

OuterContiguous reduction appears in the backward pass **primarily** as the bias gradient of the Transformer's `Linear` layers: the upstream gradient $\partial\mathcal{L}/\partial y$ of shape $(B{\cdot}T) \times F$ is summed over the $B{\cdot}T$ token dimension to produce the length-F bias gradient $\partial\mathcal{L}/\partial b_j = \sum_i \partial\mathcal{L}/\partial y_{ij}$. In the 124M configuration, this reduction is issued 49 times per micro-batch. 48 of these calls compute the bias gradients for the attention and feed-forward projections across widths $F \in \{768, 2304, 3072\}$, while the 49th call reduces the B dimension to compute the T × F (1024 × 768) positional embedding gradient. Like all reductions it is overwhelmingly *memory-bound* - it reads the entire $(B{\cdot}T) \times F$ gradient and writes only F values, with one floating-point add per element. Naive reductions lose performance to four effects: per-element index arithmetic on the strided access pattern, low instruction-level parallelism from the serially dependent accumulate chain, idle SMs when a few wide outputs cannot fill the grid, and global-atomic contention when the reduced axis spans multiple blocks.

Compile-Time Layout Specialisation. The kernel (`unified_reduce_kernel`) carries the memory layout as a non-type template parameter `PATH`, resolving the three reduction geometries into three separate branch-free specialisations at compile time: `InnerContiguous` (reduction over the last axis), `OuterContiguous` (reduction over the leading axis - the bias-gradient case), and a `Generic` stride-based fallback. The bias gradient takes the `OuterContiguous` path: the output column j is the *contiguous innermost* dimension and the reduced axis is strided by `row_stride` $= F$, so each element access reduces to a single `base_ptr[idx * row_stride]` index - with no N-D offset division or modulo. This is the key structural divergence from a general-purpose reduction, which must evaluate an `OffsetCalculator` per element, which replaces integer division by the tensor shape with magic-number multiplication Granlund and Montgomery [1994], to support arbitrary strides.

Instruction-Level Parallelism. Each thread holds `VT0 = 4` independent accumulators `acc[0..3]` and consumes four strided elements per loop iteration, keeping four reduce operations in flight before the dependent `combine`, so the latency of the accumulate chain is hidden behind issue throughput rather than exposed Volkov [2016]. The four partials are folded into `acc[0]` with a register-only combine, and a scalar tail loop handles the remainder when the reduced count is not a multiple of $4 \cdot$ `stride`.

Hierarchical Map-Reduce. The thread-local partials are reduced in stages matched to the hardware Harris [2007]. `block_x_reduce` performs the intra-warp reduction with `__shfl_down_sync` butterfly shuffles across offsets $16, 8, 4, 2, 1$ - register-to-register with no shared-memory traffic - followed by a single shared-memory exchange of the warp leaders; `block_y_reduce` folds the partials held by the y-dimension threads through shared memory. When the reduced axis is too long for one block to cover efficiently, a host-side solver enables a multi-CTA global reduce: each block stages its block-reduced partial into a scratch buffer, a *single* `atomicAdd` per output tile elects the last arriving block as the combiner (a "last-CTA" semaphore), and only that block reads the staged partials back and produces the final value. This confines cross-block communication to one atomic per output tile and one `__threadfence`, rather than the per-element global atomics a naive multi-block reduction would issue.

Launch and Occupancy Tuning. All kernel arguments - the operator functor, the resolved `GpuReduceConfig`, the source and destination pointers, and the optional staging buffers - are packed *by value* into a single `ReduceOp` struct passed as one kernel parameter, minimising `cudaLaunchKernel` argument overhead. The kernel carries `__launch_bounds__(NT, MinBlocksPerSM⟨NT⟩)`, where the blocks-per-SM target is derived from Ada's 1536-thread-per-SM limit, and the block geometry (`block_width` ≤ 32, `block_height`) together with the multi-CTA split factor are chosen by a host-side configuration solver from the problem shape before launch, so no occupancy decision is made on the device.

Table 10: Reduce kernel - bias-gradient sum over $B \cdot T = 16384$ rows, per output width F (RTX 6000 Ada). Latency is the per-call average; bandwidth columns to be completed from isolated Nsight Compute runs.

Kernel	rows	F (cols)	latency(μs)	HBM GB/s
BluBridge	16384	768	29.3	–
Pytorch	16384	768	58.0	–
BluBridge	16384	2304	191.0	–
Pytorch	16384	2304	195.8	–
BluBridge	16384	3072	338.8	–
Pytorch	16384	3072	256.6	–

4.6 Adam Optimizer

The AdamW parameter update is not a single large kernel but a sweep over *every* parameter tensor in the model - in the 124M-parameter GPT-2 configuration this is roughly 150 separate tensors ranging from a 768-element LayerNorm bias to the 50304×768 token-embedding matrix. Each tensor performs an identical, arithmetically trivial update: two exponential-moving-average accumulations and one parameter step. The update reads four arrays (g, p, m, v) and writes three (m, v, p) per element - 28 bytes of memory traffic for roughly 12 floating-point operations, an arithmetic intensity of ≈ 0.43 FLOP/byte. The operation is therefore overwhelmingly *memory-bound* and, in a naive one-kernel-per-tensor implementation, *launch-bound*: the kernel launch overhead of $\sim$150 microscopic launches per step dominates the useful work. Our SM89 implementation attacks both problems with a single multi-tensor kernel and a chunk-based load-balancing scheme, and our profiling confirms the result reaches the hardware memory ceiling.

Multi-Tensor Horizontal Fusion. The kernel (`multi_tensor_adam_sm89_kernel`) processes up to `MAX_TENSORS_PER_LAUNCH = 48` parameter tensors and `MAX_BLOCKS_PER_LAUNCH = 320` thread blocks in a *single* launch, collapsing the $\sim$150 per-tensor launches of a naive optimizer into a handful of launches per step and amortising launch overhead to near zero. The pointer tables, sizes, and block-to-work maps for all fused tensors are carried in an `AdamLaunchMetadata` struct that is passed **by value** as a kernel argument - residing in the constant bank - so no `cudaMemcpyAsync` call, no pinned staging buffer, and no asynchronous-copy fence is required to stage the metadata. Host-side overhead per launch is thus $O(1)$ in the data size. The companion optimizer-epilogue kernels - `multi_tensor_zero` (gradient reset, dtype-agnostic 16-byte `uint4` STG.128 stores) and `multi_tensor_scale` (gradient-clipping rescale) - reuse the identical metadata and chunk-mapping framework, so the entire epilogue (zero $\rightarrow$ grad-norm $\rightarrow$ clip-scale $\rightarrow$ Adam step) shares one launch-balanced design.

Chunk-Based Load Balancing. Each tensor is partitioned into fixed chunks of `CHUNK_SIZE = 32768` floats (512 threads $\times$ 4 `float4` lanes $\times$ 16 unrolled steps), and the host fills the `block_to_tensor` and `block_to_chunk` maps so that every thread block is assigned exactly one chunk of equal work regardless of which tensor it belongs to. A 768-element bias and a 38.6-million-element embedding matrix therefore contribute commensurate block counts, and all 142 Ada streaming multiprocessors stay saturated - eliminating the tail effect where a few large tensors or many tiny ones would otherwise leave SMs idle. When a tensor spans more chunks than the per-launch block budget, the host emits successive launches, each packing the next 320 chunks.

Vectorised Memory-Bound Update. Because the operation is bandwidth-bound, minimising transaction count is decisive. The main loop loads g, p, m, v as 128-bit `float4` vectors and writes m, v, p as `float4`, processing four elements per thread per iteration at the maximum LDG.128/STG.128 transaction width. Chunk boundaries that do not align to 16 bytes are handled by a scalar head and scalar tail bracketing the vectorised body, so the bulk of every chunk stays vectorised without a correctness hazard at the edges. Both moment updates use the fused multiply-add intrinsic `fmaf` - $m' = \mathrm{fmaf}(\beta_1, m, (1-\beta_1)g)$ - issuing one instruction with a single rounding step, and the bias-correction terms $\hat{b}_1, \hat{b}_2$ are precomputed once on the host, removing all per-element `powf` evaluation. The kernel carries `__launch_bounds__(256, 2)`, capping register allocation so the occupancy target of at least two resident blocks per SM is guaranteed on Ada.

Numerical Robustness. The kernel implements the decoupled-weight-decay (AdamW) update $p \leftarrow p - \mathrm{lr}\,(\hat{m}/(\sqrt{\hat{v}} + \varepsilon) + \lambda p)$ with ε placed *inside* the denominator. This ordering is essential for sparse parameters: when a token-embedding row receives no gradient on a given step its second moment is $\hat{v} = 0$, and a reciprocal-square-root applied before adding ε would produce $\mathrm{rsqrt}(0) = +\infty$ and a subsequent $0 \cdot \infty = \mathrm{NaN}$ that would poison the entire parameter. With ε in the denominator the update collapses to zero for zero-variance rows, preserving training stability.

PyTorch Baseline. PyTorch's fused AdamW path dispatches the `multi_tensor_apply_kernel` instantiated with `FusedAdamMathFunctor` (`aten/src/ATen/native/cuda/fused_adam_impl.cu`, `fused_adam_utils.cuh`, and `MultiTensorApply.cuh`). It shares the same core ideas - horizontal multi-tensor fusion, chunk-per-block assignment, `kILP = 4` vectorised load/store, and FMA-based moment updates - but is a fully general implementation that additionally carries mixed-precision dispatch (`fp16`/`bf16` parameters with `float` master copies), automatic-mixed-precision `grad_scale`/`found_inf` handling, a `capturable` code path for CUDA-graph capture, and the optional `amsgrad` variant. These branches add per-element and per-launch overhead that our FP32-specialised, sm89-tuned kernel does not pay. Both implementations are bandwidth-bound, so the operation is correctly a near-tie at the memory roofline; our kernel matches a mature production implementation while carrying strictly less control overhead.

Profiler Evidence. Nsight Compute confirms the design is at the hardware limit. A representative 320-block launch of `multi_tensor_adam_sm89_kernel` on the RTX 6000 Ada reaches **89.89% of DRAM speed-of-light** (**818.9 GB/s**) while the SM (compute) throughput is only **3.95%**, the unambiguous signature of a memory-bound kernel operating at its bandwidth ceiling. Achieved occupancy is 36.5% against a register-limited theoretical 66.7%; this does not bottleneck performance because a bandwidth-saturated kernel reaches peak throughput well below full occupancy, and the stall profile is dominated by long-scoreboard (global-memory) latency rather than execution-pipe contention - exactly the expected behaviour when DRAM is the limiter.

Table 11: Adam (AdamW) optimizer step, 124M-parameter model (RTX 6000 Ada). Latency is the measured per-step optimizer cost; effective bandwidth is the aggregate over all parameter tensors. The isolated large-tensor launch reaches 818.9 GB/s (89.89% of DRAM speed-of-light) in Nsight Compute.

Kernel	params	latency/step (ms)	eff. HBM GB/s	% peak BW
BluBridge	124M	4.45	780	81
Pytorch	124M	4.50	771	80

5 Parallelization Strategy

This section is a placeholder for the distributed parallelization strategy used in Blu-Train. The training setup uses distributed data parallelism (DDP) across eight RTX 6000 Ada Lovelace GPUs, with each GPU processing an independent mini-batch shard before synchronizing gradients across workers. This keeps a full model replica on every device while increasing effective throughput by parallelizing the data dimension.

For the 124M-parameter GPT-2-style configuration, each GPU processes a local mini-batch of $B = 16$ sequences with context length $T = 1024$. The global batch is 524,288 tokens, which gives four gradient accumulation steps under the eight-GPU DDP setup. The final version of this section will describe the exact gradient synchronization schedule, how accumulation is coordinated with optimizer steps, and how communication overhead interacts with the custom CUDA kernels.

The final write-up should also discuss implementation details such as process placement, all-reduce behavior, overlap between computation and communication, and any constraints imposed by GPU memory capacity or interconnect bandwidth. This will clarify which parts of the observed speedups come from single-GPU kernel improvements and which parts depend on scaling behavior across the full eight-GPU training run.

6 Experiments

7 Discussion

8 Conclusion

References

Torbjörn Granlund and Peter L. Montgomery. Division by invariant integers using multiplication. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation (PLDI)*, pages 61–72. ACM, 1994.

Mark Harris. Optimizing parallel reduction in CUDA. NVIDIA Developer Technology, 2007.

Vasily Volkov. *Understanding Latency Hiding on GPUs*. PhD thesis, University of California, Berkeley, 2016.